COMP2040 – Operating Systems

Lab 03: CPU Scheduling

Overview

This lab turns the Week 4 lecture into numbers you can check. You will trace two preemptive policies by hand, then complete a small simulator for FCFS, SJF and round robin (RR) and use it to compare policies on the same workload.

Rules (as in the lecture): one CPU, one CPU burst per process, no I/O, zero switching cost, times in ms. $T = C - A$, $W = T - B$, $R = S - A$ (arrival A, burst B, first start S, completion C). Ties go to the earlier arrival, then the lower process number; a new arrival preempts only if it is *strictly* better. In RR, a process that arrives during a quantum joins the queue before the process whose quantum just ended.

Process	P1	P2	P3	P4	P5
Arrival / ms	0	1	3	5	16
Burst / ms	5	3	6	1	2
Priority	3	2	1	2	1

The **lab workload**. A smaller priority number means a higher priority, as in the lecture.

The starter file `sched_sim.c` already contains this workload (`lab`) and the lecture's shared workload (`lecture`). Build it once, and again after each change:

```
gcc -std=c11 -Wall -Wextra sched_sim.c -o sched_sim
```

Submission

Upload two files to Canvas before the deadline shown there: `StudentID_Lab03.pdf`, a short report, and `StudentID_Lab03.zip`, your completed `sched_sim.c`. For each task, give the commands, the relevant output (as text) and short answers to the questions.

1 Trace preemptive schedules by hand

The simulator does not implement SRTF or priority scheduling, so trace them on paper. For the lab workload, draw the Gantt chart and compute T, W and R for every process, with averages, under (i) SRTF and (ii) preemptive priority.

Include in your report.

(a) Both Gantt charts, including any idle time, and both metric tables.
(b) At which times does a preemption happen under each policy, and why?
(c) Which process waits longest under priority scheduling? How would aging help?

2 Run FCFS

FCFS is already implemented. Run it on both workloads; the `lecture` output must match the FCFS slide of Lecture 4 (mean waiting 7.00 ms).

```
./sched_sim fcfs lecture
./sched_sim fcfs lab
```

Include in your report.

(a) The output for the `lab` workload.
(b) The trace shows the CPU idle from 15 to 16 ms. Why, if P5 has not finished?
(c) Which process suffers most from the convoy effect here, and why?

3 Shortest job first

Complete `pick_sjf()` (`TODO 1`): among the processes that have *arrived* and are not finished, choose the one with the shortest burst. `./sched_sim sjf lecture` must give mean waiting 5.50 ms.

```
./sched_sim sjf lab
```

Include in your report.

(a) The output for the `lab` workload.
(b) Why does SJF run P1 first, although P1 is not the shortest job?
(c) Compare the mean waiting time of SJF with your SRTF result from Task 1. What does preemption gain, and what can it cost a long job?

4 Round robin and context switches

Complete `schedule_rr()` (`TODO 2`) using the queue functions in the starter, and the context-switch counter in `run_slice()` (`TODO 3`). A context switch is counted each time the CPU starts a *different* process from the one that ran last; the first dispatch is not counted. `./sched_sim rr lecture 2` must give mean waiting 5.00 ms and mean response 2.00 ms. Then run the lab workload with quanta 1, 2, 4 and 8:

```
./sched_sim rr lab 1
```

Include in your report.

(a) A table of quantum against mean waiting, mean response and context switches.
(b) Explain how response time and context switches change with the quantum. For which quantum does RR give the same schedule as FCFS, and why?
(c) If every switch cost 0.1 ms, how would CPU utilisation and response time pull the choice of quantum in opposite directions?